

PR-Driven Self-Service GitOps for OpenChoreo Developers

Project 702 proposal

Mentor

Vajira Jayasrimal

Project type

Design research and implementation

Duration

6 months, 1 contributor

Initial scope

GitHub, Flux CD, ReleaseBinding promotion

Summary

OpenChoreo can deploy applications in two ways that do not meet. Portal actions write straight to the cluster, which leaves no Git history or review. GitOps keeps everything in Git, but a developer has to hand-write each manifest, even for a routine promotion. The `occ` CLI can already generate the manifests, but nothing connects it to a self-service UI path.

This project closes that gap. A developer action in the portal (create a component, deploy to an environment, promote a release) will generate the exact OpenChoreo custom resource, open a pull request against the GitOps repository, and let Flux or Argo CD apply it after merge. The proposal also delivers the design work for showing pending changes, surfacing conflicting in-flight changes, configuring repository layout, and handling credentials across Git providers.

Problem and motivation

- **No audit trail from the portal.** Direct API calls change the cluster without review or history.
- **Manual YAML in GitOps.** Every promotion needs a hand-authored ReleaseBinding committed by the developer.
- **Tooling not connected.** `occ` generates manifests, but only the Argo Workflow in `openchoreo/sample-gitops` uses that to open PRs, and only as part of a build.
- **Known generation failure.** Manifest generation breaks for repositories that contain only application-level resources, because platform resources (ComponentTypes,

Traits) are missing.

- **Unanswered design questions** raised in GitHub Discussion #3100: pending changes, layout, credentials, and concurrent edits.

Background and related work

Pattern	What it offers	Gap for OpenChoreo
Backstage built-in SCM actions	<code>publish:github:pull-request</code> creates a branch, commits workspace files and opens a PR. Inputs include <code>update</code> , <code>draft</code> , <code>reviewers</code> and <code>createWhenEmpty</code> . Equivalents exist for GitLab, Bitbucket and Azure DevOps.	Nothing OpenChoreo-specific. No conflict awareness, no manifest rendering.
OpenChoreo build workflows (<code>sample-gitops</code>)	Argo Workflows build an image and open a PR with <code>Workload</code> , <code>ComponentRelease</code> and <code>ReleaseBinding</code> . A bulk workflow opens promotion PRs.	Triggered by builds, not by a developer action in the portal. Layout is fixed.
Red Hat Developer Hub with Argo CD	Portal templates open PRs that Argo CD syncs.	To be researched in Phase 1 and compared against OpenChoreo's resource model.

One constraint from the controller work in openchoreo PR #4885: promotion-path enforcement is done in the controller, not an admission webhook, so a GitOps commit that updates several environments can still converge. A bad promotion therefore merges and appears as a resource condition. The portal should validate the path before opening a PR.

Goals and non-goals

Goals	Non-goals
Portal actions produce reviewable PRs instead of direct cluster writes.	Automatic merge-conflict resolution.
Reuse <code>occ</code> manifest generation rather than a second generator.	A replacement for the existing build workflows.
Layout driven by configuration.	Full multi-provider implementation (GitLab and others are researched and designed, built only if time allows).
Developers can see where a change	Changes to OpenChoreo controllers.

sits between PR and running resource.
Conflicts are surfaced before a second
PR is opened.

Proposed architecture

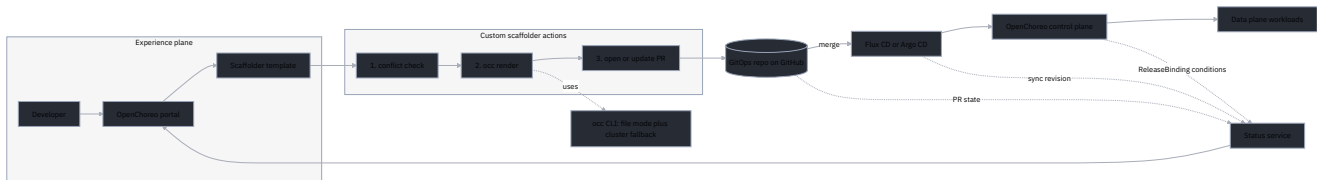


Figure 1. End-to-end architecture. Solid lines are the write path; dotted lines are the status read path.

Request flow

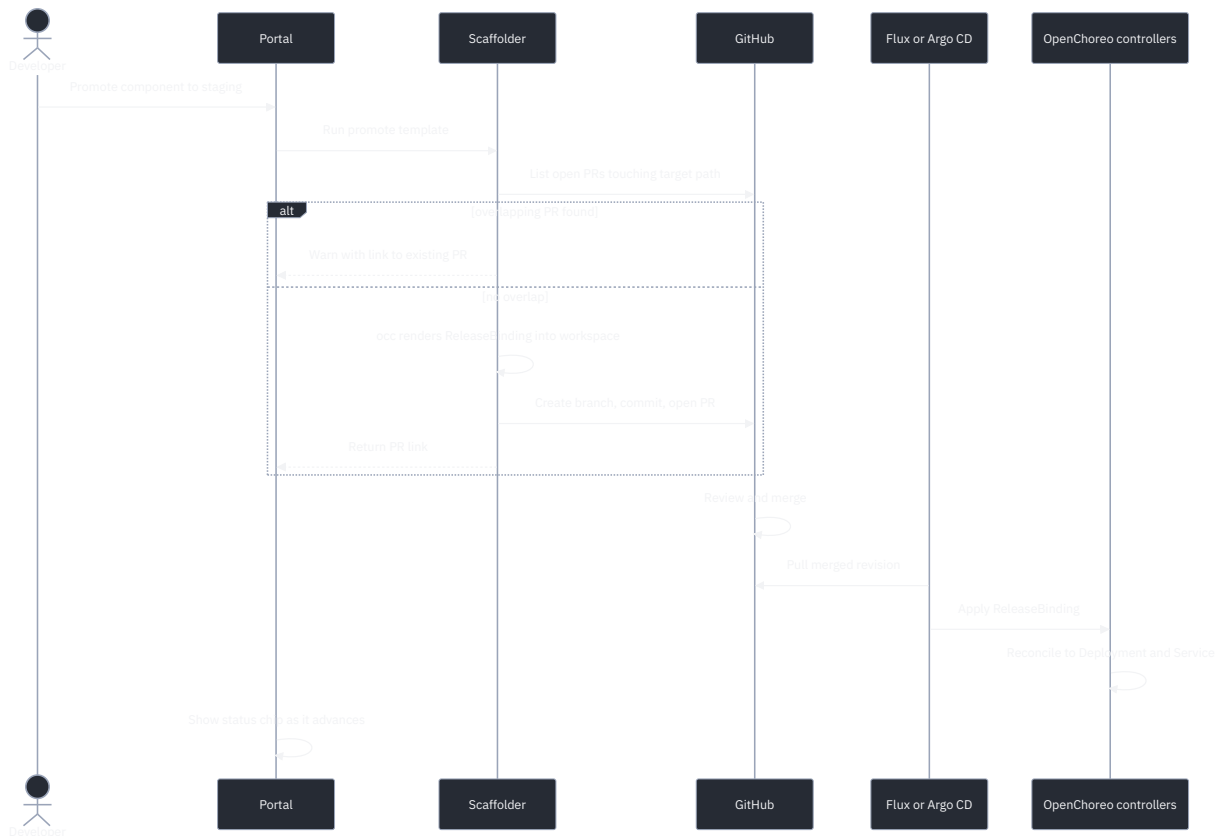


Figure 2. Promotion flow, including the conflict gate.

Design

Scaffolder actions and templates

Three pieces, composed in templates for create-component, deploy and promote:

- `openchoreo:gitops:check_conflicts` : lists open PRs and compares their changed

- `openchoreo:gitops:check-conflicts` : lists open PRs and compares their changed files with the target path.
- `openchoreo:gitops:render` : runs `occ` against a checkout of the GitOps repo and writes manifests into the workspace. It uses Backstage's shell helper for the sub-process and streams output to the task log.
- The built-in `publish:github:pull-request` to commit and open the PR, with a deterministic branch name, `update: true` and `createWhenEmpty: false`.

Resource order: ReleaseBinding promotion first, then ComponentRelease and Workload, then component creation. `occ` needs to be present in the Backstage backend image. How the OpenChoreo portal registers custom backend actions will be confirmed against the `backstage-plugins` repository in the first weeks.

Configurable repository layout

Target paths come from pattern strings, with the layout used by `sample-gitops` as the default. Because every resource has a deterministic path, the same resolver also powers conflict detection.

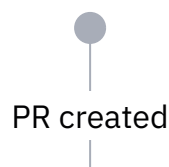
```
openchoreo:
  gitops:
    repo: github.com?owner=acme&repo=platform-gitops
    baseBranch: main
    layout:
      releaseBinding: "namespaces/{namespace}/projects/{project}/components/{co
      release:         "namespaces/{namespace}/projects/{project}/components/{co
      workload:        "namespaces/{namespace}/projects/{project}/components/{co
```

Hybrid dependency resolution in occ

Today, generation can fail when the repository holds only application resources. The change adds a lookup order: repository files first, then the live cluster for ComponentTypes, Traits and their cluster-scoped variants. The fallback is explicit (a flag or mode), the output says which source resolved each dependency, and it ships with unit tests and an upstream issue discussed with the mentor before the PR.

Showing pending changes

A change passes through systems that each know only their own stage:



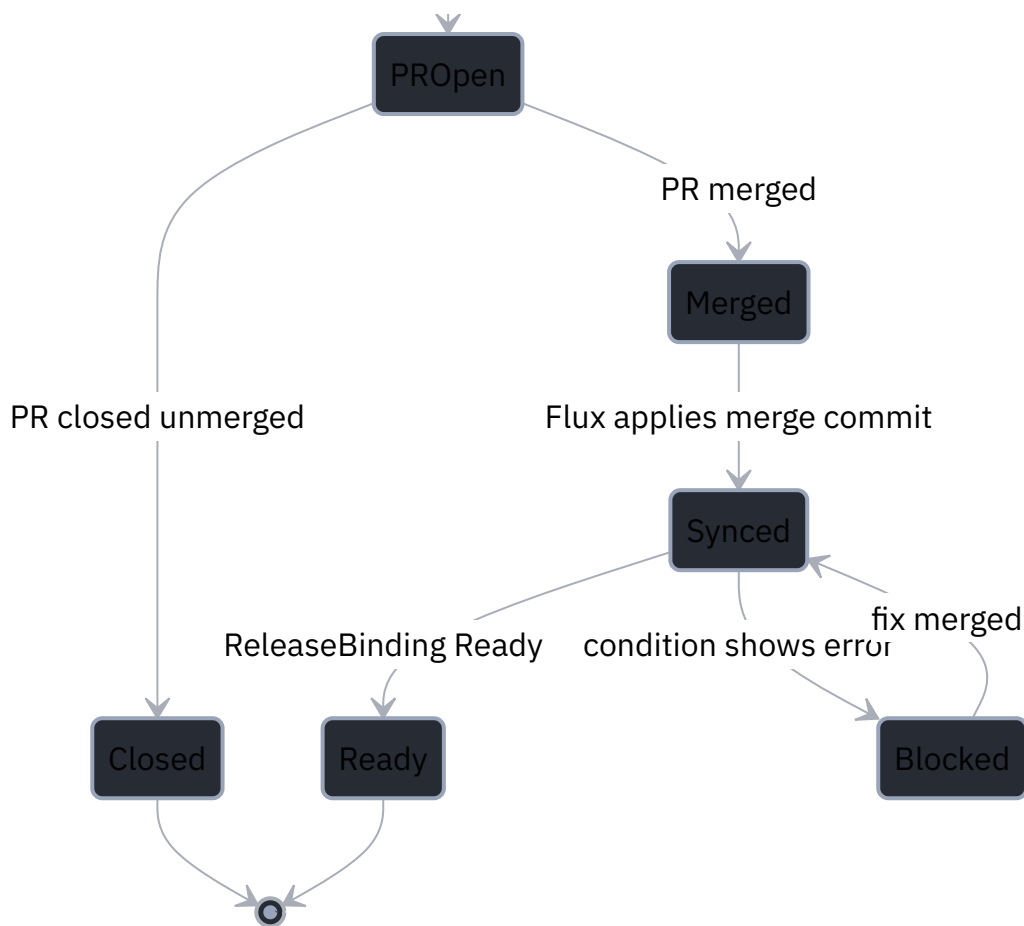


Figure 3. Change lifecycle shown to the developer.

Stage	Read from
PR open, in review, closed	GitHub API through Backstage's SCM integration
Merged, not yet synced	Flux GitRepository or Kustomization revision compared with the merge commit
Synced, reconciling, ready	ReleaseBinding conditions <code>ReleaseSynced</code> , <code>ResourcesReady</code> , <code>Ready</code>

PRs are tied to resources through the branch naming convention plus a label or PR-body marker holding kind, namespace, project, component and environment. A small status service combines the three sources. A status chip on the environment card is the proof of concept; the full UI remains a design.

Surfacing conflicting changes

Before any PR is opened, the conflict check looks for two cases: an open PR whose changed files include the target path, and a merged change to the same resource that has not synced yet. When found, the developer sees the other PR, its author and state, and chooses to view it, update it, or cancel. The system detects and warns. It never resolves merge conflicts

automatically.

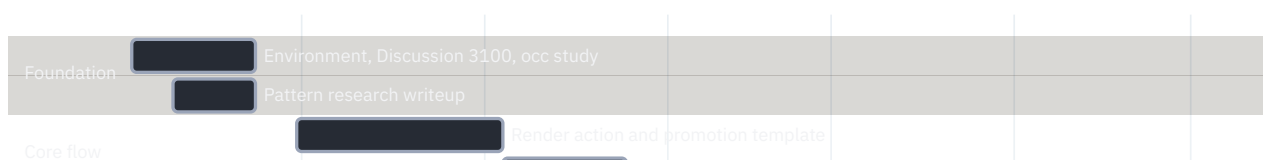
Git providers and credentials

The research compares reuse of Backstage's SCM integrations and credential providers (GitHub App token, user OAuth token, personal access token) and the GitLab equivalents. The core decision is PR authorship: the user (better attribution, needs sign-in scopes) or a bot (simpler, loses attribution). Secrets stay in Backstage configuration or Kubernetes secrets, never in the GitOps repo. GitHub is implemented; GitLab is documented as a design.

Deliverables

Deliverable	Output
Research and comparison of self-service GitOps patterns	Written comparison and recommendation, answering Discussion #3100
Scaffolder actions and templates	Actions and templates for ReleaseBinding promotion, then ComponentRelease, Workload, create-component
Configurable layout model	Pattern-based resolver with sample-gitops default, tested against a second layout
Hybrid resolution in occ	Go change with tests, merged or under review upstream
Pending-change design	Design document and status-chip proof of concept
Conflict design	Design document and working conflict check in the actions
Provider and credential research	Comparison across GitHub, GitLab and others
Docs and end-to-end sample	Portal action to merged PR to running resource, with a recorded demo

Timeline



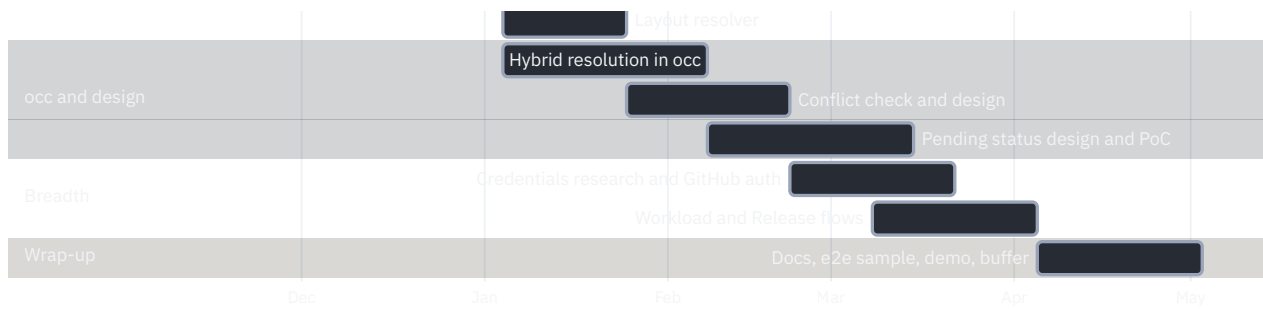


Figure 4. Indicative schedule; shift to the actual start date.

Testing and success criteria

- Unit tests for actions (Backstage mock action context) and for `occ` hybrid resolution.
- Integration tests on a k3d cluster with Flux against a throwaway GitHub repository, using dry-run mode where possible.
- End-to-end: promote in the portal, PR appears with the expected file, merge, Flux syncs, ReleaseBinding reaches Ready.
- Two simultaneous promotions of the same component: the second is warned before a PR is opened.
- Success means a developer promotes a release without writing YAML, every change is reviewable in Git, and the portal shows its stage at all times.

Risks

Risk	Mitigation
Unknown mechanism for adding custom actions to the OpenChoreo portal	Inspect <code>backstage-plugins</code> in week 1; raise with mentor early
<code>occ</code> and CRDs change quickly	Pin versions; discuss Go changes with maintainers before coding
Scope growth	Guaranteed core is GitHub plus ReleaseBinding promotion; GitLab, Argo CD and full UI are stretch
Template misuse leaking files into PRs (past Backstage advisory)	Keep Backstage updated; restrict who registers templates
Argo CD has no OpenChoreo tutorial yet	Build on Flux; add Argo CD only if time remains

Open questions

- What do Discussion #3100 participants already prefer for PR authorship and pending-state display?
- Should manifest generation stay in the Backstage backend, or move behind an OpenChoreo API later?
- Where should the status service live: a Backstage backend plugin or an OpenChoreo component?

This proposal is based on the project description, the OpenChoreo repositories and docs, and Backstage's published documentation. The `occ` command flags and the portal's action registration have not yet been verified against the CLI reference and the Backstage plugins documentation; the first two weeks confirm both.